CNNs e invariância no MNIST

Breve apresentação do script

`mnist_cnn_invariancia_augmentation.py`

> **Objetivo:** preparar experimentos controlados para verificar se uma CNN mantém a classificação correta quando imagens do MNIST sofrem deslocamento, rotação, ruído, mudança de contraste ou uma combinação dessas transformações.

O código fornece a base de dados, as transformações, os dataloaders, uma CNN inicial e funções de treinamento e avaliação. Ele pode ser importado por outro script ou executado diretamente para verificar os tamanhos dos batches.

1. Pergunta do projeto

CNNs aprendem padrões locais e usam compartilhamento de pesos e pooling, mas isso não garante invariância completa. O experimento compara o desempenho no MNIST original com versões visualmente alteradas.

Invariância	
O resultado deveria permanecer igual depois da transformação.	
Robustez empírica	
O desempenho permanece aceitável em uma transformação específica.	
Equivariância	
A representação acompanha a mudança de posição ou forma de maneira	

previsível.

Hipótese

Augmentation deve melhorar as transformações vistas ou semelhantes durante o treino.

2. Reprodutibilidade

A função `seed_everything` fixa as sementes do Python, NumPy e PyTorch. Também configura o comportamento determinístico da GPU quando ela estiver disponível.

```
seed_everything(seed=42)
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
```

Isso torna as comparações entre condições mais justas e facilita repetir o experimento.

3. Transformações de teste

Conjunto	Transformação	Pergunta
original	Nenhuma	Qual é a acurácia de referência?
translated	Deslocamento fixo de 5×3 pixels	O pooling tolera mudança de posição?
rotated	Rotação fixa de 20 graus	O modelo reconhece o mesmo dígito inclinado?

noisy	Ruído gaussiano	A decisão depende de detalhes frágeis?
contrast	Brilho 0,60 e contraste 1,90	A normalização preserva o desempenho?
combined	Deslocamento + rotação + ruído + contraste	As perturbações acumuladas provocam falhas?

As classes `FixedTranslate`, `FixedRotate`, `AddGaussianNoise` e `FixedBrightnessContrast` tornam os testes determinísticos, permitindo comparar exatamente as mesmas condições.

4. Treino com e sem augmentation

Sem augmentation

`ToTensor → Normalize`

O modelo vê apenas imagens MNIST em sua aparência original.

Com augmentation

`ToTensor → RandomAffine → ruído opcional → Normalize`

O modelo recebe variações de posição, escala, cisalhamento e ruído durante o treino.

Os dois dataloaders usam o mesmo dataset base, batch size configurável e

embaralhamento no treinamento. Os seis dataloaders de teste não embaralham os exemplos.

5. Modelo inicial

```
Conv2d(1, 16, 3, padding=1)
ReLU → MaxPool2d(2)
Conv2d(16, 32, 3, padding=1)
ReLU → MaxPool2d(2)
Flatten → Linear(32×7×7, 64)
ReLU → Linear(64, 10)
```

A entrada possui uma imagem em escala de cinza com 28×28 pixels. Os dois max-poolings reduzem as dimensões para 7×7. A saída possui dez logits, um para cada dígito.

6. Fluxo de treinamento

1. Selecionar CPU ou GPU.
2. Carregar MNIST e criar os dataloaders.
3. Instanciar a CNN.
4. Usar Adam com learning rate 0,001 e CrossEntropyLoss.
5. Executar o forward pass, calcular a loss, fazer backpropagation e atualizar os pesos.
6. Avaliar a acurácia nos conjuntos original e transformados.

```
optimizer.zero_grad()
logits = model(x)
loss = criterion(logits, y)
loss.backward()
optimizer.step()
```

7. Execução

```
pip install torch torchvision matplotlib numpy
python mnist_cnn_invariancia_augmentation.py
```

Para executar a demonstração de treinamento, descomente uma das chamadas `demo_train_and_evaluate` no final do arquivo. O MNIST será baixado automaticamente para `./data`.

8. Como interpretar os resultados

A acurácia no conjunto original é a referência. Para cada transformação, deve-se observar a diferença em relação ao mesmo modelo no teste original.

Resultado observado	Interpretação
Queda pequena	Robustez empírica para aquela transformação.
Queda grande	A decisão depende de padrões sensíveis à transformação.
Augmentation melhora	O modelo aprendeu a lidar melhor com variações semelhantes.
Augmentation piora o original	Existe um possível custo de especialização ou regularização.

Uma conclusão tecnicamente correta deve evitar afirmar que a CNN é automaticamente invariante. O termo mais adequado costuma ser **robustez parcial e dependente da transformação**.

9. Conclusão

> O script cria uma estrutura reproduzível para testar a hipótese central do projeto. CNNs podem apresentar alta acurácia no MNIST original e ainda falhar com pequenas mudanças de posição, rotação, ruído ou contraste. O data augmentation é uma forma prática de ensinar ao modelo quais variações devem ser tratadas como equivalentes.

robustez.